Computer Communications 154 (2020) 111–117

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

# The views, measurements and challenges of elasticity in the cloud: A review

Ahmed Barnawi [a], Sherif Sakr [b], Wenjing Xiao [c,*], Abdullah Al-Barakati [a]

[a] King Abdulaziz University, Jeddah, Saudi Arabia
[b] University of New South Wales, Sydney, Australia
[c] Huazhong University of Science and Technology, Wuhan, China

## ABSTRACT

Elasticity is one of the most important characteristics of cloud computing paradigm which enables deployed application to dynamically adapt to a changing demand by acquiring and releasing shared computational resources at runtime. Thus, elasticity is a key enabler for economies of scale in the cloud that enhances utility of cloud services. In practice, elasticity requires an autonomous management to reduce the gaps between the demand and supply of the computing resources at runtime. In this article, we Provide a review of the approaches and techniques from different perspectives. We examine elasticity management aspects from macro and micro economics perspectives to support value-driven elasticity decisions. We analyze the different views for measuring the elasticity of cloud-based systems. Furthermore, we discuss some of the open challenges in this domain.

## 1. Introduction

Elasticity has been always represented as an essential feature of cloud computing that enables deployed applications services to quickly adapt to a dynamic workloads by acquiring and releasing shared computational resources at runtime. This feature represented the cornerstone for economies of scale in the cloud and enhances utility of cloud services. Initially, the term was ambiguous and there was no common and clear definition and understanding for it. Several efforts have attempted to address these challenges by focusing mainly on the context of Cloud Computing; however, these efforts have differed in the angle they looked at this feature. For example, Armbrust et al. [1] defined Elasticity as the Cloud's ability to add and remove computing resources at a "fine grain" level and with a lead time of minutes so that the supplied capacity can closely follow the workload demand. With the term "fine grain", they point to the granularity of one server at a time; however, they have not discussed anything about at what granularity the supplied server capacity should be provided. According to the NIST definition of Cloud Computing, Elasticity represent the rapid provisioning and de-provisioning capability of virtually infinite resource capacity with unconstrained purchasable quantity at any single moment. Other efforts have focused on defining Elasticity through the lenses of the *Cloud Consumer*. According to this perspective, three key aspects of Elasticity of a Cloud platform can be identified as follows [2]:

- **The speed at which the computing resources are provisioned and deprovisioned.** Ideally, resources should be available at the very instant they are requested for so that the applications hosted in the Cloud do not suffer from service disruption. Similarly, resources should be deprovisioned at the very moment they are not needed to avoid excess payment.
- **Maximum amount of resources that can be purchased at once.** To the Consumer, the amount of resources in the providers' resource pool should appear to be virtually infinite and they should be able to acquire as many servers as they need to meet their workload demand.
- **The granularity of usage accounting.** This means that the de-provisioning aspect of elasticity is not only a function of the speed to release a resource, but also depends on whether charging for that resource is immediately ceased or not. However, the current state of the Cloud is not so ideal and some providers (e.g., AWS, Google App Engine, etc.) impose an hour-long charging quanta to the consumer.

An Elastic system is capable of dynamically adjusting its resource capacity in the face of fluctuating workload conditions, thus maintaining a predefined/tolerable performance threshold in the delivered service with minimal operational expenditure. In particular, two main features of the Cloud work as the main motivational factors behind an Elastic system: Allocation or De-allocation of resources on demand and Pay-per-use Pricing model. In practice, an Elastic system starts with some minimum allocation of resources so that it can process requests under normal workload condition. In the course of time, there might arise situations (e.g., seasonality of user demand, ash crowds etc.) when the system resources tend to saturate because of a sudden increase in the arrival of requests. In this condition, an Elastic system stretches its capacity by allocating additional resources from the Cloud to avoid service disruption or performance degradation. When the request rate falls from the peak value to the normal, some resources in the system remain

* Corresponding author.
*E-mail addresses:* ahmed@kau.edu.sa (A. Barnawi), ssakr@cse.unsw.edu.au (S. Sakr), wenjingx@hust.edu.cn (W. Xiao), al-barakati@kau.edu.sa (A. Al-Barakati).

https://doi.org/10.1016/j.comcom.2020.02.010
Received 9 December 2019; Received in revised form 15 January 2020; Accepted 3 February 2020
Available online 12 February 2020
0140-3664/© 2020 Elsevier B.V. All rights reserved.

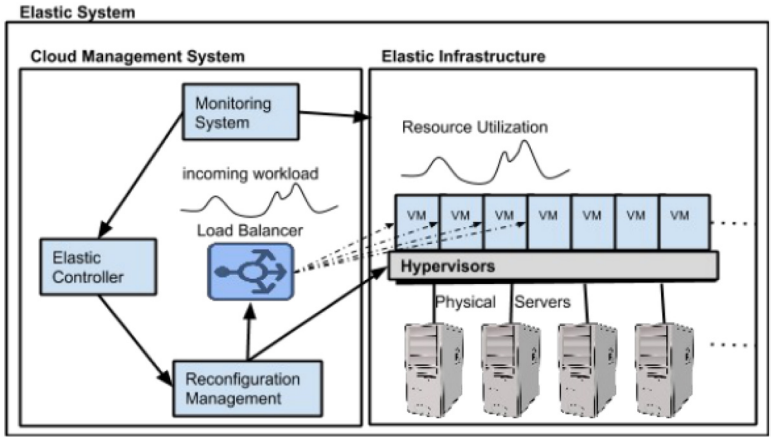



**Fig. 1.** High Level architecture of an elastic system.

under-utilized. In this condition, the system contracts its capacity by de-allocating a portion of resources to the Cloud, thus not paying any more for idle utilization. This is how an Elastic system meets its performance criteria with minimal cost in response to workload fluctuations.

Fig. 1 illustrates the basic architecture of an Elastic system. At the core of the Elastic system, there are two components : a *scalable infrastructure* and a *management system*. The Cloud providers offer infrastructures or development platforms to the consumers in the form of virtual machines (VM) with access to network bandwidth and storage. A *hypervisor* or Virtual Machine Monitor (VMM) abstracts out virtual machines from the underlying physical hardware. The hypervisor has control over physical server's resources and can manage allocation (de-allocation) of resources to the virtual machines in case a scale-up (scale-down) is required. In some cases, Cloud consumers prefer to distribute their application load to a pool of virtual machines with dynamic scale-out (scale-in) capability. This task is performed by a *load-balancer* which sits in front of the pool of virtual machines and forwards the incoming requests to the attached VMs based on a pre-configured routing algorithm (e.g., round-robin, least outstanding requests etc.). The *scalable infrastructure* is administered by a Cloud Management System which is comprised of a *load-balancer*, a *monitoring system*, a *reconfiguration management* module and in most cases, an *elastic controller*. The monitoring system periodically monitors the resource utilization and performance characteristics of the VM instance pool. It can also send notification to the elastic controller or the application administrator once an alarm is configured. An alarm watches a single metric (e.g., CPU utilization, Latency etc.) over a particular interval and sends notification once the observed value of the metric breaches a pre-defined threshold for a specific number of times. The elastic controller, on receiving the alarm notification, makes adjustment to the resource capacity by invoking the reconfiguration management API. The reconfiguration management service places request to the scalable infrastructure for an allocation (de-allocation) of resources and updates the load-balancer's instance pool, if necessary, once the request gets fulfilled. This is how an Elastic System can adapt to workload demand by stretching and contracting its resource capacity on demand. The quality of the adaptation process depends on several parameters, for example, provisioning and de-provisioning time of the scalable Cloud infrastructure, supplied resource granularity, Cloud provider's pricing quanta, elastic controller's adaptation strategy to load variation etc.

Several research efforts [3,4] have considered the above aspects of the consumer perspective. For example, Islam et al. [3] introduced another dimension to Elasticity that can be termed as *precision* or *resource granularity* - i.e., how closely the provided resource follows the demand in terms of utilization. It is intuitive that if the resources are provided with the finest granularity in no time, the demand and supply curves will coincide with each other and the cloud platform is considered as absolutely elastic. The reality of Cloud Elasticity, however, is not as perfect as the absolute one; so the demand and supply rarely coincide with each other, thus resulting in either excess or insufficient capacity. Herbst et al. [4] further defined elasticity in terms of the speed of provisioning/deprovisioning and precision of provided resource granularity; however, they skip the granularity of charging quanta and its impact on billing in their proposed Elasticity definition, perhaps, because of their attention to that of the technical aspects only. In this article, we analyze the different perspectives and views for defining the properties of elastic cloud-based software systems. In addition, we analyze the different views for measuring the elasticity of cloud-based systems. Furthermore, we discuss some of the open challenges in this domain.

## 2. Elasticity vs. Scalability and Efficiency

The terms Elasticity, Scalability and Efficiency are commonly used (and often misused) in the context of Cloud Computing. In general, Scalability can be defined as the degree to which a system is able to meet application specific performance criteria in response to larger workloads by incrementally adding proportional amount of resource capacity [5]. In traditional client–server systems, applications are designed for scalability to make sure that the operational expenses grow linearly with respect to workload demand. When workload requirements are reduced to a normal level, resource deletion is usually not considered, because the purchased resource is already sunk cost. However, in the context of the Cloud and its pay-as-you-go pricing model, the financial motivation for releasing resources during low demand cannot be ignored and hence the notion of Scalability needs to be further refined. In Cloud Computing, Scalability describes the degree to which the system is able to meet application specific performance objectives in response to varying workload demand by adjusting its resource capacity [6]. Resource capacity can be scaled in two different ways: Vertical scaling (also known as, scaling up/down) and Horizontal scaling (also known as, scaling out/in). Vertical scaling means how resources can be added to (removed from) an existing computing node. On the other hand, Horizontal scaling refers to addition (removal) of computing nodes to an existing cluster. It should be noted, however, that scalability has no time dimension and hence it does not capture how quickly the system adapts to a changing workload demand over time. In contrast, time is one of the core aspects of Elasticity which depends on the speed of response to fluctuating workload demand. Moreover, Scalability does not consider how frequently and at what granularity the system adjusts its resource capacity as the workload demand varies over time. On the contrary, Elasticity is concerned about how precisely the supplied capacity follows the workload demand over time; therefore, these aspects are considered as crucial ingredients in defining and measuring Elasticity.

**Table 1**
Comparison: Elasticity, Scalability and Efficiency.

| Aspect | Elasticity | Scalability | Efficiency |
|---|---|---|---|
| Adaptability | x | x | |
| Scaling time | x | | |
| Resource granularity | x | | |
| Cost | x | x | x |

Efficiency describes a system's ability to process a certain amount of work with the smallest possible effort. This term can be applied either to part of a system, e.g. to a single resource, or to an entire system. Weber et al. [6] classified efficiency into three categories: Cost efficiency, Energy efficiency and Resource efficiency. The workload for which efficiency is measured could be either constant or fluctuating over time. In contrast, for constant workloads, Elasticity is not an attractive option as there is no need for resource adaptation in this case; so the value of Elasticity can be best realized in a fluctuating demand setting. Table 1 summarizes the aspects of comparison between Elasticity, Scalability and Efficiency features of software systems.

## 3. Properties of elastic software systems

In general, an elastic system is a system which is capable of dynamically adjusting its resource capacity in the face of fluctuating workload conditions. Thus, it maintains a predefined/tolerable performance threshold in the delivered service with minimal operational expenditure. In particular, two main features of the Cloud work as the main motivational factors behind an Elastic system: allocation or de-allocation of resources on demand and Pay-per-use Pricing model. In practice, an Elastic system usually starts with some minimum allocation of resources so that it can process requests under normal workload conditions. In the course of time, there might arise situations (e.g., seasonality of user demand, flash crowds, etc.) when the system resources tend to saturate because of a sudden increase in the arrival of requests. In this condition, an Elastic system stretches its capacity by allocating additional resources from the Cloud to avoid service disruption or performance degradation. As the peak is over and the request arrival rate drops back to normal, some resources in the system remain under-utilized. In this condition, the system contracts its capacity by de-allocating a portion of resources to the Cloud, thus not paying any more for idle utilization. This is how an Elastic system meets its performance criteria with minimal cost in response to workload fluctuations. In principles, the properties of an elastic system can be classified into three categories: elasticity, resource management and quality of service.

### 3.1. Elasticity

- **Eagerness**: describes the speed of adaptation of a system in response to workload variation over time.
- **Sensitivity**: denotes the minimum change in workload that triggers an adaptation. This parameter of an elastic system is defined over a range on the recent measured load; if the load stays within this limit, then the system resources are sufficient to meet the demand and hence no adaptation action needs to be triggered; otherwise, the system needs to undergo an adaptation process to prevent resource saturation or idle utilization.
- **Plasticity**: defines a system's inability to release resources and go back to the minimal resource configuration from a state with increased resource capacity. Ideally, an elastic system should be able to de-allocate a portion of its resources and revert back to the minimal configuration within a reasonable amount of time once the load intensity decreases. Systems which fail to exhibit this property are called plastic.

### 3.2. Resource management

- **Precision**: defines the accuracy with which the elastic system can allocate or de-allocate resources so that the difference between the supplied resource and workload demand is minimal.
- **Oscillation**: An elastic system is said to exhibit an oscillatory behavior when it repeatedly allocates and de-allocates resources even when the workload demand is stable. Even though oscillation appears to be a valid behavior, it has a negative impact on the running cost for the application hosted in the Cloud.
- **Resource Thrashing**: refers to a situation when elastic systems tend to exhibit opposite adaptations in a very short time duration; for instance, a system may acquire additional resources in one interval, and then when the resource allocation phase is completed, it starts to release these resources. Resource thrashing is a temporary, yet quick oscillation in the adaptation of resources. During resource thrashing situation, the portion of resources taking part in oscillation cannot perform any useful work, however, they do increase the running cost of the Cloud-hosted application.
- **Cool-down period**: is the amount of time that must elapse before a new scaling action can be issued after triggering an adaptation. During this interval, the elastic controller inhibits itself from triggering any scaling action in order to let the system stabilize after the previous adaptation. Designers usually set up this cool-down period in order to prevent the system from adapting faster than the provisioning/de-provisioning delay of the Cloud platform.
- **Bounded concurrent adaptations**: is the maximum number of scaling actions that are allowed by the elastic controller when an adaptation is in progress. It can be viewed as a relaxed generalization of the cool-down period strategy where the maximum number of scaling actions during an adaptation is one.
- **Bounded resource usage**: is a property that allows the elastic system to use resources beyond a certain threshold, if necessary, for a pre-specified time interval. This means, whenever the elastic controller allocates more resources than specified by some temporary threshold, it should de-allocate those excess resources within a pre-defined time interval.

### 3.3. Quality of service

- **Bounded QoS degradation**: is a property that constrains the amount of QoS degradation during adaptation. It conveys the fact that the normally-required QoS threshold can only be stratified when the elastic system is stable, i.e., it is not going through any adaptation process; otherwise, a more relaxed level of QoS threshold needs to be enforced during adaptation.
- **Bounded actuation delay**: constrains the maximum delay introduced by the elastic controller and reconfiguration service to execute a scaling action. Intuitively, the time required for the application to be ready to serve requests must be limited by this value. Table 2 summarizes the research on elasticity managemen.

## 4. Elasticity measurement and benchmarking

Elasticity measurement and benchmarking is mainly concerned with the measurement of systems hosted on the Cloud, seeking to measure its sustained Elastic performance as well as revealing the potential strengths and weaknesses of the underlying Cloud environments, adaptive mechanisms, design choices and deployment configurations. In general, elasticity measurement techniques can be split into two distinct categories: macro-benchmarking frameworks and micro-benchmarking frameworks.

**Table 2**
Overview of elasticity management approaches.

| Approach | SLA-awareness (Quality) | Cost-awareness (Economics) |
|---|---|---|
| Nouri et al. [7] | Y | Y |
| Ralha et al. [8] | Y | N |
| Carlos et al. [9] | N | N |
| Bauer et al. [10] | Y | N |
| Bitsakos et al. [11] | Y | N |
| Nikravesh et al. [12] | Y | Y |
| Pascual et al. [13] | Y | Y |
| Liu et al. [14] | Y | N |
| da Rosa et al. [15] | Y | Y |
| Ficco et al. [16] | Y | Y |
| Khebbeb et al. [17] | Y | Y |
| Rodrigues et al. [18] | Y | Y |
| Trihinas et al. [19] | Y | Y |

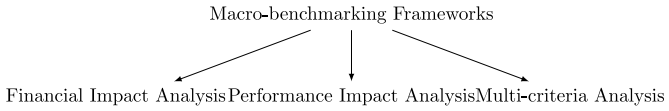



**Fig. 2.** Taxonomy of macro-benchmarking modeling approaches.

### 4.1. Macro-benchmarking frameworks

A Macro-benchmarking framework quantifies the Elasticity of the system as a whole. It intends to measure Elasticity with respect to a specific class of applications and reflects the Elasticity of the overall system under study. it is particularly useful to the interested stakeholders in that it helps them draw conclusions and informed comparisons over competitive Cloud offerings, adaptive policies, design alternatives and deployment configurations. It should be noted, however, it does not reveal the reasons behind the system's poor Elasticity behavior.

The frameworks in this category can be classified based on their respective approaches to model Elasticity behavior of the system. Fig. 2 provides an overview of this categorization which are described as follows:

#### 4.1.1. Financial impact analysis

These frameworks usually take on a stakeholder's perspective and hold the assumption that the stakeholder's application has to meet a predefined SLA while minimizing operational expenses. In particular, these approaches [3,20] quantifies the implications of under-provisioning and over-provisioning in terms of monetary cost. Failure to meet the SLA for insufficient resources or excess payment for unutilized resources contributes to the penalty which is further transformed into monetary cost. The obvious merit of these frameworks lies in modeling the stakeholder's detriment in monetary terms; however, their results may turn out to be obsolete with the change in the provider's pricing scheme. For example, Weinman [20] coined the term "Penalty model" approach which explicitly takes on the consumer's viewpoint. The main idea behind Weinman's model stems from the basics of economics, i.e., demand and supply. In his measurement model, the resource demand is expressed as a function over time, $D(t)$ and allocated resource supply is expressed as a function over time, $R(t)$. Suppose that, for some time point $t_i$, there is a difference between the observed demand $D(t_i)$ and provided resource supply $R(t_i)$. If $D(t_i) > R(t_i)$, then the system is under-provisioned, i.e., the current resource capacity is not adequate to meet the demand. On the other hand, if $D(t_i) < R(t_i)$, then the system is over-provisioned, i.e., the resource capacity is under-utilized at time $t_i$. Weinman characterizes "Perfect Capacity" as $D(t_i) = R(t_i)$ for all time point $t_i$. This model measures "Imperfect Elasticity" in terms of financial implications to the consumer. To encapsulate this view, the under-provisioned intervals are associated with opportunity cost which is linear in the quantity of insufficient resources and the over-provisioned intervals are associated with cost which is also linear in the amount of idle resources but assumed to be cheaper than the former. Weinman proposes a financial loss function to combine the under-provisioning and over-provisioning costs and express Elasticity as a single numeric value for the duration of a workload execution. This framework presents a workload suite consisting of several patterns: constant, linearly growing, exponential growth and random ones with uniform distribution and random walks. Aslanpour et al. [21] presented a cost-aware approach for autoscaling that executes scale-down commands via aware selection of surplus virtual machines. Therefore, surplus virtual machines can be kept quarantined for the rest of their billing period in order to maximize the cost efficiency. Casalicchio et al. [22] presented an energy-aware auto-scaling algorithms for Cassandra virtual data centers which aims of reducing the energy consumption or resource usage in general in a way that can heavily reduce the reliability of Cassandra in term of the consistency level offered. Bitsakos et al. [11] presented an approach for using Deep Reinforcement learning techniques to achieve automatic elasticity by automatically deciding whether to request/release virtual machines inside a NoSQL cluster according to user-defined policies/rewards. Rigi et al. [15] presented a proactive elasticity model that combines cloud and high performance computing to address the IoT scalability problem by using Autoregressive Integrated Moving Average (ARIMA) and Weighted Moving Average to predict the IoT load behavior. Nikravesh et al. [12] presented a cost-driven decision maker that focuses on the cloud client's cost preferences and uses genetic-based algorithm to configure a rule-based system to minimize the total auto-scaling cost. Fokaefs et al. [23] presented an approach for formalizing the opportunities for economically optimal scaling that takes into account not only the cost of infrastructure but also the revenue from service delivery and eventually the profit of the service provider.

#### 4.1.2. Performance impact analysis

The frameworks of this category are specifically concerned about the performance implications for imperfect Elasticity on the application or the database cluster [24,25]. They measure Elasticity in terms of the performance overhead caused by the prolonged adaptation interval. They are useful to understand and predict the performance of the elastic system; however, the Elasticity scores computed by them are susceptible to fluctuations because of the performance variability of Cloud environments. For example Dory et al. [24] were one of the early pioneers to consider the problem of Elasticity measurement for distributed NoSQL databases and developed a framework that quantifies Elasticity as a dimensionless measure. They view Elasticity as the reaction of a database cluster when new nodes are added to it or existing nodes are removed from it under some workload. The authors intend to encapsulate two main aspects in their Elasticity metric: the cluster's stabilization time right after adding or removing a node and performance overhead (i.e., amount of deviation from stable performance) during the stabilization period. In order to keep it simple, they restrict their scope to the scale-out phase only. They hypothesize that right after the addition of a new node to a cluster, the execution time will rise initially and then gradually settle back to the normal level as the new nodes are up and running. They argue that no matter how small the stabilization period is, the standard deviation of execution time will increase because of the data transfer overhead between nodes during this interval. This approach keeps the workload intensity constant during the stabilization period. It does not address use case scenarios where the workload intensity is fluctuating over time; overlooking these scenarios introduce the risk of wrongly estimating Elasticity for database clusters. Another shortcoming of this work is that it does not specify any metric to account for the contraction phase of the database cluster; so the stakeholders are supposed to have only a partial view of Elasticity. These concerns, if not properly addressed, might jeopardize the adoption of this framework. Rodrigo et al. [26] presented an approach for automatic resource elasticity for

high performance applications in the cloud without user intervention or source code modification. Pascual et al. [13] presented an approach for evaluating to what extent the reduction of the time required by VM management operations, namely deployment and shutdown, impacts the performance of applications and the behavior of reactive and proactive auto-scaling policies. The experiments of this work showed that short management times drastically reduce the use of resources, while allowing the application to operate within the required performance bounds.

#### 4.1.3. Multi-criteria analysis

Multi-criteria Analysis approach facilitates rational decision-making in situations where multiple conflicting criteria are present [27]. In the context of Cloud, the stakeholders often have conflicting goals; this approach tends to identify the stakeholder's goals of interest and evaluate how the elastic system can meet these goals in terms of a combined utility measure. The main concern with this approach lies in the appropriate selection of the preference criteria and whether those criteria (e.g., performance, cloud pricing scheme, etc.) are consistent over time. The performance variability and frequent pricing scheme update are two factors that might affect the benchmarking results obtained by this approach. For example, Majakorpi [27] proposed a measurement model that applies the Multi-criteria analysis to transform conflicting preference objectives into an Elasticity metric for a workload. This work holds the assumption that an application's business objectives can be expressed in terms of low level metrics, such as, performance measures, costs, etc. In this approach, Elasticity is viewed as a meta-quality — a quality that is related to other qualities of the system. For this reason, this work formalizes the preference objectives based on application specific business requirements, e.g., performance, cost, etc. and determines how much the application conforms to these preference criteria. These preference factors most often conflict with each other; for example, a gain in performance achieved with additional resources might oppose the cost objective and vice versa. Therefore, this work applies Multi-criteria analysis technique to define a utility function that quantifies the perceived utility of the system as the weighted combination of achieved conformance under multiple preference objectives. The QoE metric is then computed by aggregating the achieved utility over a particular interval and normalizing that value for unit interval. This framework is perfectly valid to measure the Elasticity of the application in cases when there are multiple conflicting preference objectives. Nevertheless, it offers weak guidelines on workload suite design; the workloads considered here are purely arbitrary and lacks realistic features, e.g., seasonality, occasional trend, bursts, etc. Furthermore, it does not provide any direction on how to combine the QoE of multiple workloads into a single summary measure, thereby making it more difficult for the stakeholders to have a conclusive view and fair comparison across the competitive options. Chen and Bahsoon [28] presented a self-adaptive decision making approach for autoscaling in the cloud. The approach is designed to adaptively produce autoscaling decisions that lead to well-compromised trade-offs without heavy human intervention by leveraging ant colony inspired multi-objective optimization for searching and optimizing the trade-offs decisions. Alfonso et al. [9] presented a framework to deploy elastic virtual clusters that can dynamically changed to satisfy both the user application's computing requirements and to minimize the amount of energy consumed by the underlying physical cluster. Massimo et al. [16] presented a meta-heuristic approach for cloud resource allocation based on the bio-inspired coral-reefs optimization paradigm with respect to cloud provider's optimization objectives, as well as customer requirements. All the requirements of both parties are expressed through Service Level Agreements formalized by using a fuzzy linguistic method.

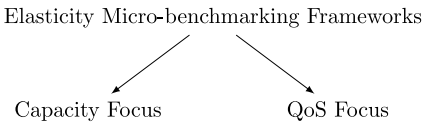



**Fig. 3.** Taxonomy of micro-benchmarking modeling approaches.

### 4.2. Micro-benchmarking frameworks

A micro-benchmarking framework measures either specific aspects of Elasticity or a specific portion of the Elastic system. It aids in the investigation of bottlenecks for imperfect elasticity and provides the stakeholders with insights on how to tweak the system for improved levels of elasticity. This framework, however, has a more detail-oriented view and does not offer any yardstick to draw conclusions or comparisons over a range of alternative elastic platforms, adaptive mechanisms, design options and deployment configurations. The frameworks falling into this category can be further sub-classified based on their modeling approach into the following categories (Fig. 3):

#### 4.2.1. Capacity focused micro-benchmarking framework

This category specifically concentrates on the capacity related measures of the system under study [6]. It includes VM provisioning and de-provisioning delay, maximum number of VMs that can be provisioned during the peak, precision of supplied resource capacity, temporal distribution of the reconfiguration points and so on. The frameworks belonging to this category are suitable to diagnose potential issues for imperfect elasticity; nevertheless, they fail to characterize the impact of elasticity on the application or database. For example, Coutinho et al. [29] presented an approach that concentrates specifically on Elasticity analysis based on allocation time and resource usage and propose a set of secondary metrics to further investigate the system's orchestrated adaptability behavior from the micro level. This measurement model characterizes Elasticity behavior in terms of seven metrics: Under-provisioned allocation time, Over-provisioned allocation time, Stable allocation time, Transient allocation time, Total under-provisioned allocated resource, Total over-provisioned allocated resource and Total stable allocated resource. Under-provisioned allocated time and Over-provisioned allocation time correspond to the interval used in carrying out resource allocation and de-allocation operations respectively. Stable allocation time refers to the interval when the allocated resources are stable, that is, the system is not going through any adaptation process. Transient allocation time refers to the interval when the system makes the transition from one state to another right after an allocation or de-allocation operation is executed. Total under-provisioned allocated resources and Total over-provisioned allocated resources are defined as the total resource quantity over all under-provisioned states and all over-provisioned states respectively. Total stable allocated resources correspond to the resource quantity over stable time periods. This model, however, has some conceptual limitations; it does not capture over-provisioning during stable and under-provisioned periods. In addition, it interprets transient allocation time as a theoretical metric but could not justify its presence in relation to practical scenarios. In particular, this metric reflects the underlying Cloud platform's overhead in configuring the load-balancer, propagating the resource configuration changes to the DNS, etc. Moreover, it requires further refinement to diagnose pragmatic issues, e.g., the difference between the charged and allocated resource quantity during adaptation intervals.

#### 4.2.2. QoS focused micro-benchmarking framework

This approach tends to quantify the QoS impact of Elasticity on the application or database during the adaptation phase [30]. These measures are useful to understand the interactions of Elasticity with

the application or database during the adaptation interval; nevertheless, they do not necessarily reveal the underlying reasons of a poor Elasticity behavior in terms of system level measures. For example, Cooper et al. [30] developed an open source benchmarking framework, Yahoo! Cloud Serving Benchmark (YCSB), to facilitate fair evaluation of data serving systems in the cloud. This benchmark has two tiers: performance and scaling tier. The scaling tier specifically concentrates on measuring the impact of Elasticity. It defines two metrics: *Scaleup* and *Elastic speedup*. The scaleup metric assesses whether the database cluster can serve an increased amount of workload with a graceful performance (e.g., latency) as more nodes are added to it. This is a reflection of database scalability. The other metric, elastic speedup, measures the performance (read latency) of the database cluster during the adaptation period of node addition and data re-balancing under constant workload. This metric does not consider the use case scenario where the database cluster needs to contract itself by releasing a portion of nodes, thereby reflecting a partial view of Elasticity. In addition, the workload profiles for elasticity measurement do not resemble characteristics of real-world traces. Gandhi et al. [31] presented an approach that automatically scales the infrastructure to meet the user-specified performance requirements, even when multiple user requests execute concurrently. The proposed approach is designed to automatically learn the system parameters for each application so that it proactively scale the infrastructure to meet performance guarantees without requiring any offline application profiling or benchmarking, training data, or expert knowledge. Sahni and Vidyarthi [32] presented a heterogeneity-aware approach for auto scaling which adapts to the dynamic workload while maintaining the Quality of Service (QoS) by useing online profiling of Cloud resources and workload history to analytically estimate subsequent resource requirements. The proposed approach used a heuristic-based mechanism to provision the smallest set of different size resource configurations in order to meet the QoS targets at reduced cost and improved resource utilization. Evangelidis et al. [33] presented a techniques which is based on performance modeling and formal verification to produce performance guarantees on particular rule-based auto-scaling policies. The framework supports cloud application with the ability of conducting formal reasoning on the configuration of their auto-scaling policies and minimize QoS violations. Liu et al. [14] presented an approach that uses fuzzy logic to automatically and adaptively adjusts thresholds and cluster size for a web application with a main focus on reducing resource consumption without violation of Service Level Agreement (SLA).

## 5. Challenges and open issues

Although there has been much interest around adaptive resource management and designing cloud-based elastic systems, there remains a number of unresolved challenges and open issues. We highlights some of these issues as follows:

- **Quasi-immediate scaling latency**: In practice, resource allocation and de-allocation in the cloud is not immediate; it takes several minutes to resize the resource capacity on-demand. This non-negligible scaling latency may not be adequate to support real-time and mission-critical applications operating at the finest timescale of milli-seconds or micro-seconds. For instance, high frequency trading applications need to rapidly adapt the resource capacity (in milli-seconds or micro-seconds interval) in order to carry out transactions at a very high speed in response to market-sensitive events and announcements. However, the current state of elasticity is not so mature to ensure acceptable QoS for such applications. More sophisticated adaptive scaling strategies should be developed to assure good elasticity for these applications.
- **Limitation in the max scaling factor**: One of the promises of Cloud Computing is that consumers can expect to have virtually infinite resource capacity on demand from an elastic cloud service. However, the maximum attainable resource capacity that a consumer can lease at any instant of time is a finite quantity which is bounded by some default value. Configuring the default maximum resource quantity is not instantaneous; instead, it requires intervention of the support center. The maximum attainable threshold for the default maximum resource capacity is also strictly restricted by the cloud provider.
- **Lack of fine-grained control**: Most of the web and e-commerce workloads exhibit burstiness at the fine timescale (in the order of seconds or sub-seconds). To ensure effective elasticity for these workloads, cloud consumers require fine-grained monitoring metrics and dynamic scaling mechanisms. However, the existing cloud service APIs do not allow the consumers of cloud services to monitor the metrics and configure adaptive scaling decisions at the sub-minute granularity. This constraint poses challenge to achieving effective elasticity in response to fine-scale bursty request arrivals.
- **Unpredictable elasticity behavior**: The elasticity of an adaptive cloud system is a random quantity. The perceived elasticity across different runs of the same workload may vary in an unpredictable manner, even with the same experimental setup. Several environmental factors are responsible for the unpredictable elasticity behavior in the cloud; examples include, random scaling latency [34], performance variability effect [35], uncertainty in the responsiveness of the resource management API calls [36], etc. Cloud applications provision and de-provision resources on-demand by making API calls to the cloud provider's resource management web service. However, recent studies revealed increased uncertainty in the responsiveness of these API calls [36], including long tails in the response time distribution, significant failure rates in executing the resource management request etc. Moreover, heterogeneity in the physical cloud infrastructure and interference effect from the co-located VMs also influence the required amount of resources to serve the same workload intensity. The presence of these factors induce randomness in the elasticity of the cloud-hosted applications, thereby affecting the runtime expenses and overall revenue. The elasticity benchmarking approaches should address this unpredictability issue and come up with solutions in order to ensure repeatable and credible elasticity benchmarking results. The cloud providers could also employ improved provisioning strategies to minimize the unpredictability in the delivered elasticity of the cloud services.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## CRediT authorship contribution statement

**Ahmed Barnawi:** Project administration. **Sherif Sakr:** Methodology. **Wenjing Xiao:** Resources. **Abdullah Al-Barakati:** Visualization.

## Acknowledgments

This work was supported by the Deanship of Scientific Research (DSR), King Abdulaziz University, Saudi Arabia, Jeddah, under grant No. (RG-1-611-39). The authors, therefore, gratefully acknowledge DSR technical and financial support.

## References


[1] M. Armbrust, A. Fox, R. Griffith, A.D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, M. Zaharia, A view of cloud computing, Commun. ACM 53 (4) (2010) 50–58, http://dx.doi.org/10.1145/1721654.1721672.

[2] D. Chiu, Elasticity in the cloud, ACM Crossroads 16 (3) (2010) 3–4, http://dx.doi.org/10.1145/1734160.1734162.

[3] S. Islam, K. Lee, A. Fekete, A. Liu, How a consumer can measure elasticity for cloud platforms, in: Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering, ACM, 2012, pp. 85–96.

[4] N.R. Herbst, S. Kounev, R. Reussner, Elasticity in cloud computing: What it is, and what it is not, in: ICAC, 2013, pp. 23–27.

[5] G. Brataas, P. Hughes, Exploring architectural scalability, in: ACM SIGSOFT Software Engineering Notes, Vol. 29, ACM, 2004, pp. 125–129.

[6] A. Weber, N. Herbst, H. Groenda, S. Kounev, Towards a resource elasticity benchmark for cloud environments, in: Proceedings of the 2nd International Workshop on Hot Topics in Cloud Service Scalability (Hot-TopiCS 2014), Co-Located with the 5th ACM/SPEC International Conference on Performance Engineering (ICPE 2014), ACM, 2014.

[7] S.M.R. Nouri, H. Li, S. Venugopal, W. Guo, M. He, W. Tian, Autonomic decentralized elasticity based on a reinforcement learning controller for cloud applications, Future Gener. Comput. Syst. 94 (2019) 765–780.

[8] C.G. Ralha, A.H. Mendes, L.A. Laranjeira, A.P. Araújo, A.C. Melo, Multiagent system for dynamic resource provisioning in cloud computing platforms, Future Gener. Comput. Syst. 94 (2019) 80–96.

[9] C. de Alfonso, M. Caballer, A. Calatrava, G. Moltó, I. Blanquer, Multi-elastic datacenters: auto-scaled virtual clusters on energy-aware physical infrastructures, J. Grid Comput. 17 (1) (2019) 191–204.

[10] A. Bauer, N. Herbst, S. Spinner, A. Ali-Eldin, S. Kounev, Chameleon: A hybrid, proactive auto-scaling mechanism on a level-playing field, IEEE Trans. Parallel Distrib. Syst. 30 (4) (2018) 800–813.

[11] C. Bitsakos, I. Konstantinou, N. Koziris, DERP: A deep reinforcement learning cloud system for elastic resource provisioning, in: 2018 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), IEEE, 2018, pp. 21–29.

[12] A.Y. Nikravesh, S.A. Ajila, C.-H. Lung, Using genetic algorithms to find optimal solution in a search space for a cloud predictive cost-driven decision maker, J. Cloud Comput. 7 (1) (2018) 20.

[13] J.A. Pascual, J.A. Lozano, J. Miguel-Alonso, Effects of reducing vms management times on elastic applications, J. Grid Comput. 16 (3) (2018) 513–530.

[14] B. Liu, R. Buyya, A.N. Toosi, A fuzzy-based auto-scaler for web applications in cloud computing environments, in: International Conference on Service-Oriented Computing, Springer, 2018, pp. 797–811.

[15] R. da Rosa Righi, E. Correa, M.M. Gomes, C.A. da Costa, Enhancing performance of iot applications with load prediction and cloud elasticity, Future Gener. Comput. Syst. (2018).

[16] M. Ficco, C. Esposito, F. Palmieri, A. Castiglione, A coral-reefs and game theory-based approach for optimizing elastic cloud resource allocation, Future Gener. Comput. Syst. 78 (2018) 343–352.

[17] K. Khebbeb, N. Hameurlain, F. Belala, Modeling and evaluating cross-layer elasticity strategies in cloud systems, in: International Conference on Model and Data Engineering, Springer, 2018, pp. 168–183.

[18] V.F. Rodrigues, R. da Rosa Righi, G. Rostirolla, J.L.V. Barbosa, C.A. da Costa, A.M. Alberti, V. Chang, Towards enabling live thresholding as utility to manage elastic master-slave applications in the cloud, J. Grid Comput. 15 (4) (2017) 535–556.

[19] D. Trihinas, G. Pallis, M.D. Dikaiakos, Monitoring elastically adaptive multi-cloud services, IEEE Trans. Cloud Comput. 6 (3) (2015) 800–814.

[20] J. Weinman, Time is money: the value of on-demand, 2011.

[21] M.S. Aslanpour, M. Ghobaei-Arani, A.N. Toosi, Auto-scaling web applications in clouds: a cost-aware approach, J. Netw. Comput. Appl. 95 (2017) 26–41.

[22] E. Casalicchio, L. Lundberg, S. Shirinbab, Energy-aware auto-scaling algorithms for Cassandra virtual data centers, Cluster Comput. 20 (3) (2017) 2065–2082.

[23] M. Fokaefs, C. Barna, M. Litoiu, From devops to bizops: Economic sustainability for scalable cloud applications, ACM Trans. Auton. Adapt. Syst. (TAAS) 12 (4) (2018) 25.

[24] T. Dory, B. Mejías, P. Van Roy, N.-L. Tran, Comparative elasticity and scalability measurements of cloud databases, in: Proc of the 2nd ACM Symposium on Cloud Computing (SoCC), Vol. 11, 2011.

[25] D.M. Shawky, A.F. Ali, Defining a measure of cloud computing elasticity, in: Systems and Computer Science (ICSCS), 2012 1st International Conference on, IEEE, 2012, pp. 1–5.

[26] R. da Rosa Righi, V.F. Rodrigues, C.A. Da Costa, G. Galante, L.C.E. De Bona, T. Ferreto, Autoelastic: Automatic resource elasticity for high performance applications in the cloud, IEEE Trans. Cloud Comput. 4 (1) (2015) 6–19.

[27] M. Majakorpi, et al., Theory and practice of rapid elasticity in cloud applications, 2013.

[28] T. Chen, R. Bahsoon, Self-adaptive trade-off decision making for autoscaling cloud-based services, IEEE Trans. Serv. Comput. 10 (4) (2015) 618–632.

[29] E.F. Coutinho, D.G. Gomes, J.N.d. Souza, An analysis of elasticity in cloud computing environments based on allocation time and resources, in: Cloud Computing and Communications (LatinCloud), 2nd IEEE Latin American Conference on, IEEE, 2013, pp. 7–12.

[30] B.F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with ycsb, in: Proceedings of the 1st ACM Symposium on Cloud Computing, ACM, 2010, pp. 143–154.

[31] A. Gandhi, P. Dube, A. Karve, A.P. Kochut, L. Zhang, Providing performance guarantees for cloud-deployed applications, IEEE Trans. Cloud Comput. (2017).

[32] J. Sahni, D.P. Vidyarthi, Heterogeneity-aware adaptive auto-scaling heuristic for improved qos and resource usage in cloud environments, Computing 99 (4) (2017) 351–381.

[33] A. Evangelidis, D. Parker, R. Bahsoon, Performance modelling and verification of cloud-based auto-scaling policies, Future Gener. Comput. Syst. 87 (2018) 629–638.

[34] A. Iosup, N. Yigitbasi, D. Epema, On the performance variability of production cloud services, in: Cluster, Cloud and Grid Computing (CCGrid), 2011 11th IEEE/ACM International Symposium on, IEEE, 2011, pp. 104–113.

[35] J. Schad, J. Dittrich, J.-A. Quiané-Ruiz, Runtime measurements in the cloud: observing, analyzing, and reducing variance, Proc. VLDB Endow. 3 (1–2) (2010) 460–471.

[36] Q. Lu, L. Zhu, L. Bass, X. Xu, Z. Li, H. Wada, Cloud api issues: an empirical study and impact, in: Proceedings of the 9th International ACM Sigsoft Conference on Quality of Software Architectures, ACM, 2013, pp. 23–32.




**Ahmed Barnawi** is currently a professor of information and communication technologies at Faculty of Computing and IT (FCIT) in King Abdulaziz University (KAU). He is the managing director of the KAU Cloud computing and Big Data Research group. He acquired his Ph.D. from the University of Bradford, UK, in 2005 and his M.Sc. from UMIST (University of Manchester), UK, in 2001. Prof. Barnawi acted as an associate and Visiting Professors in Canada and Germany. Prof. Barnawi is an active researcher with good research fund awards track. His research interest include Big data, cloud computing, future generation mobile systems, advanced mobile robotic applications and IT infrastructure architecture. He published near to 100 papers in peer reviewed journals.



**Sherif Sakr** is a Professor in the Department of Health Informatics at King Saud bin Abdulaziz University for Health Sciences, Saudi Arabia. Prof. Sakr had appointments with Macquarie University (Australia), Microsoft Research (USA), Alcatel Lucent Bell Labs and Cairo University (Egypt). He received his Ph.D. degree in Computer and Information Science from Konstanz University, Germany in 2007. He received his B.Sc. and M.Sc. degrees in Computer Science from the Faculty of Computers and Information in Cairo University, Egypt, in 2000 and 2003 respectively. Dr. Sakr's research interest include the areas of graph data management, big data storage and processing in cloud computing environments. Dr. Sakr is an IEEE Senior Member.



**Wenjing Xiao** received a Bachelor degree in network engineering from North China Electric Power University (NCEPU), China in 2018. Currently, she is a bachelor-straight-to-doctorate student at the Embedded and Pervasive Computing (EPIC) lab at the School of Computer Science and Technology, Huazhong University of Science and Technology. Her research interests include cloud computing, Internet of Things, and cognitive computing, etc.



**Dr. Al-Barakati** is an associate Professor at King Abdulaziz University, Saudi Arabia. In 2015, he acted as a chairman of the Information Systems Department. He received his B.Sc. (Hons) in Computer Science in 2004, M.Sc. in Software Engineering and a Ph.D. in Computer Science in 2012 from University of Sussex, United Kingdom. His research interests revolve around the utilization Big Data and Web technologies for innovative solutions and services. He has published more than 20 peer reviewed research paper on Workflow Systems, Social Media, Big Data and Digital Heritage.